Project 1 Final Report

Murphy Jacob, Atishay Jain, and Todd Jolley

Rubric

We are tasked by a city council to design a database that manages driver records, with O(1) search, insertion based on county, retrieval of N most recent/oldest licenses, and migration for inactive drivers. This report documents the design and implementation of a solution using custom C++ data structures (no STL containers) to meet the requirements.

The project uses a hash table for O(1) direct access with two separate doubly linked lists to cover all operations needed: one sorted by license data and one per county.

System Architecture and Data Modeling

Class Diagram Overview

The class diagram is used to model the structure for the project with key classes and their relationships defined below.

Helper Classes (Composition)

- `Date` : Encapsulates day, month, and year with overloaded comparison operators and validates to ensure years are within an appropriate.
- `Address` : Stores street, city, county, and zip code
- `Ticket` : Contains a `Date` and a location (`Address`)
- `MedicalCondition` : an `enum`
- **Derived Classes:** Each derived class implements `getEmploymentType()` to return a string identifying the employment type.
    - `StudentDriver`
    - `Government`
    - `SelfEmployed`

Data Structures for Fast Access

- `DriverNode` : A structure that holds a `Driver*` and two sets of `next/prev` pointers:
  - `date_next, date_prev` : for the `DateList`
  - `county_next, county_prev` : for the `CountyList`
- `DateList` : Manages a doubly linked list of `DriverNode`'s sorted by license date.
  - Functions: `insertSorted(DriverNode*), remove(DriverNode*).` It provides `head` and `tail` pointers for forward/backward traversal.
- `CountyList` : A simple doubly linked list for one county.
  - Functions: `insertAtBeginning, insertAtEnd, insertAfter, remove.` It also maintains `head` and `tail.`
- `HashTable<K, V>` : A template class with separate chaining. It proved O(1) average-case insertion, deletion, and search used for:
  - `idMap` : maps driver ID (`std::string`) to `DriverNode*`
  - `countyMap` : maps county name to (`std::string`) to `CountyList*.`
- `Vector<T>` : A custom dynamic array template used to store frequent `Addresses` and `Tickets` inside a `Driver.`
- `inactiveDatabase` : Stores `Driver*` pointers of drivers who have been migrated (list using `Vector<Driver*>`).

Central Database Class

- `DriverDatabase`
  - `dateHead, dateTail`
  - `idMap`
  - `countyMap`
  - `inactiveDatabase`
  - `totalDrivers`
  - `public:`
    - `insertDriver(Driver* position, refID)`
    - `removeByName(driverID)`
    - `displayRecent(N)`
    - `displayOldest(N)`
    - `size()`

Composition

- **Composition:** `Driver` owns its `Date` (DOB, license), `Address`, and `Vector` `containers. DriverDatabase` owns, `idMap`, `countyMap`, and `inactiveDatabase.`
- **Inheritance:** The three types of driver classes inherit from `Driver`.

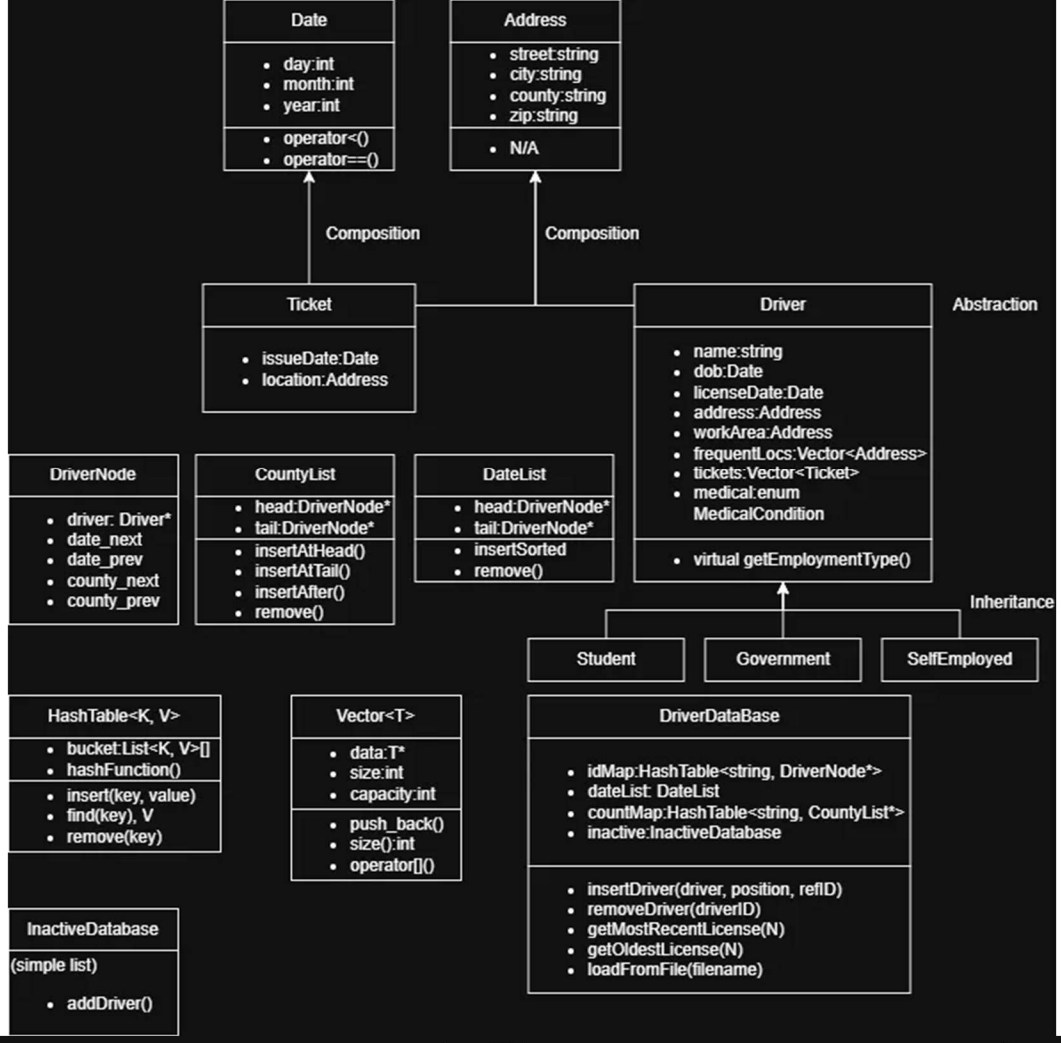
The full UML class diagram is attached as *'Project 1 UML Class Diagram.webm'*

Key Algorithms

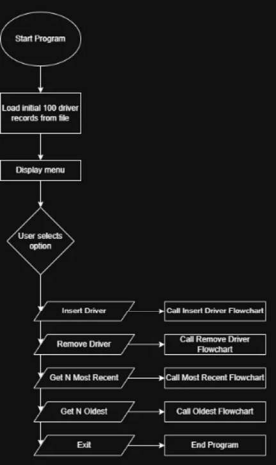
The project implements several key algorithms, each documented in an attached document (see *'Project 1 Algorithms'*) with pseudo-code.

Conclusion

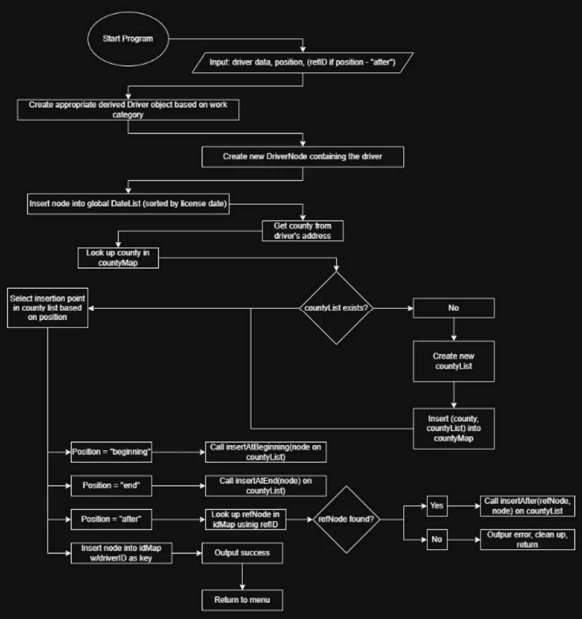
This project should meet all requirements by the rubric laid out by city council. The complete source code, Makefile, and supporting documents are included in the submission.



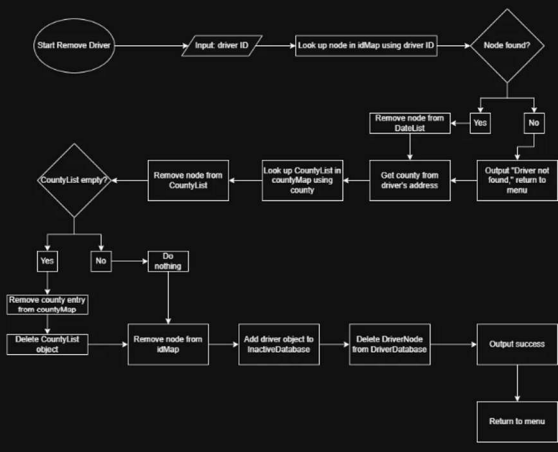
Overall System Flowchart



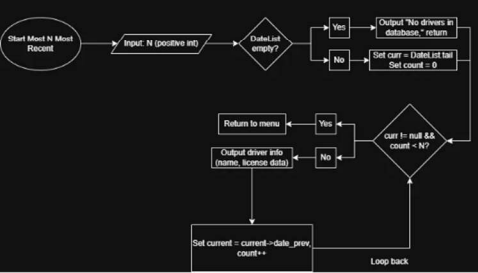
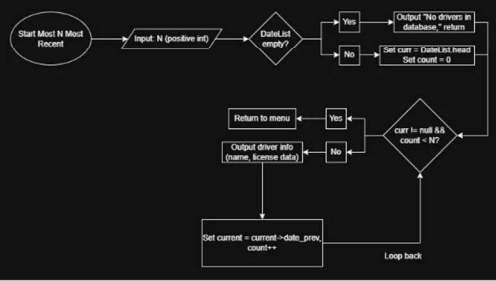
Insert Driver Flowchart



Remove Driver Flowchart



Get N Most Recent Flowchart



Get N Oldest Flowchart